Free Claude Code Plugin Brings Amazon's PR/FAQ Process to Engineers and Founders

punt-labs releases prfaq, an open-source tool to generate, import, and iterate on product discovery documents inside the terminal

Press Release

SAN FRANCISCO — Today, punt-labs released `prfaq`, a free, open-source Claude Code plugin that guides engineers and founders through Amazon's Working Backwards PR/FAQ process to evaluate product ideas before building them. The plugin provides three commands — `/prfaq` to generate a new document from scratch, `/prfaq:import` to structure and enrich an existing draft, and `/prfaq:feedback` to iterate based on pointed feedback — all inside the same Claude Code session where the user already works (see FAQ 13). Engineers who ship products at unprecedented speed can now apply the same rigor to deciding *which* products to ship.

Problem

AI coding tools have made it possible for a single person to build in a weekend what used to take a team a quarter. The bottleneck has shifted: the hard part is no longer building — it is knowing what to build and why (see FAQ 11). Builders and founders using Claude Code, Cursor, and similar tools ship fast, but most lack formal product training. They skip customer definition, skip competitive analysis, skip unit economics — and discover after burning through tokens and weeks of building that the product does not resonate.

The consequences are compounding. AI-generated code already carries its own quality burden: 66% of developers cite "almost-right" AI output as their top frustration, and 45% report that debugging AI-generated code takes longer than expected [1]. That is a code-quality problem with known mitigations. The deeper problem is strategic: many products built during the current "vibe coding" wave [2] face significant rebuilds not because the code is bad, but because the code solves the wrong problem with confidence. Enterprises already carry $1.5–2 trillion in accumulated technical debt from decades of building without sufficient discipline [3]; AI-accelerated development risks creating a new generation of the same debt, faster.

Today, a builder who wants product discipline has three options: hire a product manager they cannot afford, read a book and try to apply frameworks manually, or just iterate — ship something, get feedback, ship again. Iteration works. Some of the best products started as something a builder made for themselves, and others appreciated. But iteration has real costs: every cycle burns tokens, burns calendar time marketing to early users, and burns the founder's energy. Getting customer feedback is often the actual bottleneck. Plenty of builders would develop stronger product sense if the frameworks were more accessible — available in the workflow where they already build, not in a book or a workshop they will never attend.

Solution

`prfaq` brings product thinking directly into the builder's environment. By typing `/prfaq` in any Claude Code session, the user starts a structured conversation: Claude asks about the target customer, the problem, the competitive landscape, and the business model. It asks hard questions — the kind a product manager would ask before greenlighting a project.

From these answers, Claude generates a complete PR/FAQ document: a mock press release describing the product as if it has already shipped, followed by external FAQs (what a customer would ask) and internal FAQs (what leadership, finance, and engineering would ask). The document is evaluated against Marty Cagan's four risks framework [4] — value, usability, feasibility, and viability — producing an honest assessment of whether the product is worth building.

The output is a LaTeX-compiled PDF (see FAQ 6): a clean, professional artifact suitable for sharing with co-founders, investors, or advisors. It is not a disposable brainstorm. It is a decision-making document designed to be read, debated, and revised.

Generation is step one. `/prfaq:feedback` takes pointed direction — "focus on CTOs, not solo devs" or "the TAM is too large" — and surgically redrafts every affected section, tracing cascading effects through the document so the user iterates on substance rather than formatting. For builders who already have a draft from a workshop, a Google Doc, or their own process, `/prfaq:import` ingests the existing document, structures it against the PR/FAQ framework, researches evidence for unsupported claims, and compiles a professional output — an on-ramp that meets users where they are rather than asking them to start over. Together, the three commands form a complete product-thinking workflow inside the terminal: generate, import, and iterate.

Customer Quote

> *"I kept building MVPs that nobody used. I'd spend three weeks coding, show it to people, and get polite shrugs.* `prfaq` *made me realize I was solving my own problem, not my customer's. The PR/FAQ took forty-five minutes and saved me a month of wasted work. The hardest question it asked was 'what evidence do you have that customers want this?' I didn't have any. That was the answer."*
>
> — **Marcus Okonkwo**, Founder, SyncPilot (field-service scheduling)

Getting Started

Install with one command: `curl -fsSL https://punt-labs.github.io/prfaq/install | bash`. No account required, no subscription, no configuration. Then choose your path:

Starting fresh? Type `/prfaq`. Claude asks discovery questions about the target customer, problem, differentiation, and business model, then drafts a complete PR/FAQ document. Within an hour, the user has a compiled PDF.

Already have a draft? Type `/prfaq:import` and paste or point to an existing document — plain text, Markdown, or raw `.tex`. The plugin structures it against the framework, researches evidence for unsupported claims, and compiles a professional PDF with gap analysis.

Want to iterate? Type `/prfaq:feedback` followed by a directional instruction. The plugin traces cascading effects and surgically redrafts every affected section.

Spokesperson Quote

> "The people building products with AI are making product decisions every day — they just don't always have the frameworks. We built `prfaq` because the engineers and founders using Claude Code are also the ones deciding what to build and for whom, and they deserve a tool that meets them where they already work. The PR/FAQ process has been proven at Amazon for two decades. We didn't invent it. We made it accessible inside a terminal."

> — **Jason Freeman**, Founder, punt-labs

Call to Action

`prfaq` is free and open source, available now at `https://github.com/punt-labs/prfaq`. Install it, type `/prfaq`, and find out if your next product idea is worth building before you build it.

Frequently Asked Questions

External FAQs

Q1: What is prfaq and who is it for?

`prfaq` is a free Claude Code plugin that guides engineers and founders through the Amazon Working Backwards PR/FAQ process. The primary audience is technical builders who use AI coding tools to create products and want to validate an idea before committing weeks or months of development time. As Claude Code's user base expands beyond professional engineers [5], features like `/prfaq:import` — which enriches an existing document rather than generating one from scratch — extend the plugin's reach to non-technical users who adopt Claude Code as a thinking partner.

Q2: How is this different from using a PR/FAQ template?

A template is a blank page with headings. `prfaq` is an interactive process: Claude asks discovery questions, challenges weak answers, identifies gaps in customer evidence, and generates the document from your responses. It applies the four risks framework to evaluate your idea honestly, not just format it. A template tells you *what* to write. `prfaq` helps you figure out *what to think*.

Q3: How do I get started?

Run the one-line installer (`curl -fsSL https://punt-labs.github.io/prfaq/install | bash`), then type `/prfaq` in any Claude Code session. The plugin walks you through the entire process. No account, no API keys, no configuration. Installation takes under a minute.

Q4: What does the output look like?

A professionally typeset PDF compiled from LaTeX. The document includes a press release, external and internal FAQs, and a four-risks assessment. It is designed to be shared with co-founders, investors, or advisors — not a throwaway brainstorm document.

Q5: Do I need to know LaTeX?

No. The plugin generates the LaTeX source and compiles it automatically. You never see or edit LaTeX unless you choose to. You need a TeX distribution installed (TeX Live or similar), which the installer checks for and provides setup instructions if missing.

Q6: Why LaTeX instead of Markdown?

Three reasons. First, a PR/FAQ is a decision document — it gets shared with co-founders, investors, and advisors. LaTeX produces professionally typeset PDFs with consistent typography, precise layout, and visual credibility that Markdown-to-PDF pipelines do not match. The medium signals that the thinking is serious. Second, LaTeX's structured environments (`faqpair`, `customerquote`, `spokespersonquote`) give the LLM a precise grammar to write into — the AI reads and writes `.tex`, the human reads the compiled `.pdf`. This separation means the source format can be rich and semantic without burdening the author. Third, LaTeX's package ecosystem (biblatex for citations, mdframed for callout boxes, enumitem for structured lists) provides extensibility that Markdown lacks without custom renderers.

The trade-off is friction: LaTeX requires a TeX distribution (~4 GB), and users unfamiliar with it may find compilation errors intimidating. The installer mitigates this with automated setup, and the plugin handles all LaTeX generation — the user never writes `.tex` directly. This is a two-way door: if user feedback shows the TeX dependency is a meaningful adoption barrier, adding a Markdown output mode is straightforward since the content generation is format-independent. The structured thinking matters more than the output format.

Q7: What happens to my data?

Everything runs locally. The plugin is a set of prompt files and a LaTeX template — it does not phone home, collect telemetry, or store your data anywhere. Your PR/FAQ content is processed by Claude Code using your existing Anthropic API relationship. The generated `.tex` file and compiled PDF stay on your machine. Data handling is governed entirely by Claude Code's existing privacy model and your Anthropic terms of service.

Q8: How much does it cost?

Free. `prfaq` is open source under a permissive license. There is no paid tier, no freemium upsell, and no planned monetization. The plugin itself adds zero cost. You do need an active Claude Code subscription or API access, which is a separate cost from Anthropic.

For API users, the underlying token cost is small. A full PR/FAQ session — discovery conversation, document generation, and peer review — cumulatively consumes roughly 100–200K input tokens and 20–40K output tokens across multiple turns (context is re-sent each turn). At Sonnet pricing ($3/$15 per million tokens), that is approximately $0.60–1.20 per initial draft. Each feedback cycle is lighter: roughly 40–80K cumulative input tokens and 10–20K output tokens, or about $0.25–0.55 per round. A typical document through three or four revision cycles costs under $4 in total token spend. Opus sessions cost roughly 1.7× more.

For comparison, experienced product consultants charge $100–300 per hour depending on seniority. Claude Code subscribers on the Pro plan ($20/month), Max 5× ($100/month), or Max 20× ($200/month) pay nothing incremental — the token usage falls within the subscription's included allowance.

Q9: Can I iterate on the document?

Yes. Type `/prfaq:feedback` with a directional instruction — "reframe the problem around compliance risk" or "the TAM is too small" — and the plugin traces cascading effects across the document and surgically redrafts every affected section. Each feedback cycle recompiles the PDF and runs peer review automatically. You can also edit the `.tex` file directly or re-run the full discovery process. The Working Backwards process is designed for rapid iteration — the point is to iterate on a short document instead of iterating on a shipped product.

Internal FAQs

Value & Market

Q10: What is the total addressable market?

The addressable audience has two sides, united by a common gap: people building products with AI who lack product discipline, and experienced product thinkers who want AI leverage to operate leaner.

Side A: Non-technical builders who need product discipline. A massive wave of non-programmers is now building software. Lovable has nearly 8 million users [6]. Replit has 35M+ users, 75% of whom never write code [7]. Bolt reached $40M ARR [8]. Cursor has 1M daily active users [9]. A quarter of startups in Y Combinator's W25 cohort have codebases almost entirely AI-generated [10]. These builders ship fast but most lack any formal product training — they skip customer definition, competitive analysis, and unit economics. The "vibe coding" wave [2] is producing unprecedented build velocity with near-zero product discipline. This is the largest segment by headcount and the one with the most acute need for the frameworks `prfaq` provides.

Side B: Expert PMs seeking AI leverage. Experienced product managers and product-minded founders already know frameworks like Working Backwards [11] and four risks [4]. Their constraint is not knowledge — it is capacity. A PM at a startup wearing four hats does not have time to write a thorough PR/FAQ from scratch. An AI tool that drafts the document from a structured conversation, applies the framework rigorously, and handles the formatting lets a single PM do the work that previously required a PM plus an analyst plus a designer. This segment is smaller but higher-intent and more likely to produce documents worth sharing.

Side C: Claude Code crossing the chasm. Claude Code is the delivery vehicle. Its trajectory — surpassing $2.5B in annualized revenue [12] and doubling since January — indicates a large and growing base. Exact user counts are not public, but revenue/ARPU inference suggests an estimated 2–5M Claude Code users as of Q1 2026 (assuming a blended ARPU of $40–100/month across Pro, Max 5×, Max 20×, and API users). Claude Code is increasingly adopted by non-programmers [5], which expands the potential audience beyond the initial technical core. The trajectory matters more than the current number: if Claude Code doubles again by year-end, the denominator is 4–10M.

Distribution nuance. Claude Code is the right *architecture* for this tool: a terminal-based agent that can run structured conversations, generate LaTeX, compile PDFs, and iterate on documents is exactly the capability `prfaq` needs. But Claude Code is imperfect *distribution* for some segments. Side A users (Lovable/Bolt/Replit builders) may never install Claude Code — they build in browser-based tools, not terminals. For them, `prfaq` is reachable only if Claude Code's distribution expands to meet them (e.g., via a web interface or IDE integration) or if they migrate to Claude Code as their projects grow in complexity. Side B users (expert PMs) and technical founders are a natural fit — they already live in terminals or are comfortable adopting developer tools. The initial adoption will come from where architecture and distribution overlap: founders, engineers, and savvy PMs who already use Claude Code. The broader segments represent trajectory upside, not launch-day TAM.

This is not a TAM-based revenue argument (the plugin is free); it is a reach argument. The question is how many builders the plugin can help, not how many it can monetize. Current addressable reach via Claude Code: estimated 2–5M users, of which the product-decision-making subset (founders, technical leads, PMs) is perhaps 10–25%, yielding 200K–1.25M reachable users

today. The trajectory of both Claude Code adoption and the broader vibe-coding wave suggests this range grows significantly over 12–18 months.

Q11: What evidence do we have that customers want this?

Three categories of indirect evidence, but no primary customer data yet. First, the "vibe coding" phenomenon: thousands of products are being built at speed without product discipline, and rebuilds are already visible across startup communities and forums. Second, the explosion of "how to think about product" content targeting builders — books, courses, and newsletters suggest unmet demand for product skills among people who build with AI tools but lack formal product training. Third, the Working Backwards process itself has two decades of validation at Amazon [11], where it is mandatory for new product proposals. The gap is not whether the framework works — it is accessibility. Builders do not attend PM workshops. They install plugins. *Note: we have not yet validated that technical builders want product frameworks delivered inside Claude Code. Five to ten customer discovery interviews with engineers and technical founders would significantly strengthen this evidence base. As the non-technical audience grows [5], a separate round of interviews with that segment would inform whether* `/prfaq:import` *resonates.*

Q12: How will we validate that builders actually want this?

The customer evidence FAQ (FAQ 11) is honest: we have strong indirect evidence but no primary customer data. The validation plan is straightforward and cheap.

Phase 1: Hands-on trials (weeks 1–4). Ask 10–20 target users — engineers and technical founders who already use Claude Code — to install the plugin and use it on a real product idea. Recruit from three pools: (1) builders in the punt-labs network who have shipped side projects, (2) active posters in Claude Code community channels (Discord, Reddit r/ClaudeAI), and (3) indie hackers and technical co-founders sourced from Hacker News "Show HN" threads. The ask is simple: try `/prfaq` on your next idea and tell us what happened.

Phase 2: Opt-in metrics (concurrent with trials). With explicit user consent, participants can send anonymized usage metrics back to us: which commands were invoked (`/prfaq`, `/prfaq:import`, `/prfaq:feedback`), how many feedback cycles per document, whether the PDF was compiled successfully, and session duration. No document content is transmitted — only structural signals. This is strictly opt-in; the plugin's default is zero telemetry, and that does not change.

Phase 3: Structured feedback channel (exploratory). We are exploring a `/prfaq:feedback-to-us` slash command that lets users send structured feedback — a short form covering what worked, what did not, and whether the output was useful enough to share — via a single API call. This reduces the friction of collecting qualitative data from terminal-native users who are unlikely to fill out a Google Form. The command would be opt-in, clearly labeled, and would transmit only the user's explicit responses. This is exploratory; we build it only if the Phase 1 trial reveals that users want to give feedback but the mechanics are too cumbersome.

Signals we are looking for:

1. **Completion rate** — do users finish a full PR/FAQ, or abandon mid-session? Target: 60%+ completion among trial participants.
2. **Shareability** — do users share the generated PDF with a co-founder, investor, or advisor? Target: 30%+ of completed documents are shared.
3. **Decision impact** — did the PR/FAQ change the user's decision about what to build? Even "I decided not to build it" is a strong positive signal.

4. **Repeat usage** — do users run `/prfaq` on a second idea? Repeat usage within 30 days is the strongest signal of product-market fit for a free tool.
5. **Unprompted referral** — do trial users recommend the plugin to others without being asked?

Timeline: Recruit the first 10 trial users within 2 weeks of public launch. Complete the 10–20 person trial within 6 weeks. Synthesize findings into a revised customer evidence base. If fewer than 40% of trial users complete a PR/FAQ, or if qualitative feedback consistently indicates the output is not useful enough to share, the value risk rating should be upgraded to High and the product strategy reconsidered.

Q13: Who are the competitors and why will we win?

The nearest competitor is not a product management tool — it is the user pasting a PR/FAQ template into Claude.ai or any AI chat and having a free-form conversation. This works for experienced practitioners who already know the framework. What it lacks: structured reference guides that catch common mistakes, the four risks evaluation framework, automated peer review against cognitive biases, LaTeX compilation to a shareable PDF, and a repeatable process that improves across uses. The plugin encodes twenty years of Working Backwards practice into a reusable workflow; a blank chat starts from zero every time.

Beyond that, traditional competitors include product management tools (Productboard, Aha!, Notion templates) and business planning tools (Lean Canvas generators). None of these are integrated into the builder's working environment. They all require context-switching: leave the current tool, open a browser, fill in a form. A Claude Code plugin runs in the same session where the builder already works — the shift from "building" to "product thinking" happens in a single command, not a tab switch.

The most credible competitive response is Anthropic itself. This takes two forms, and both are worth naming honestly. First, Anthropic could feature the plugin — promote it in their plugin directory, recommend it in onboarding, or highlight it as a case study. If that happens, we get distribution we cannot buy. Second, Anthropic could build a native product-thinking feature into Claude Code. If that happens, they validate the category: the idea that terminal-based AI tools should help users decide *what* to build, not just *how* to build it. Both outcomes are positive for the product-thinking-in-terminal thesis. The strategic exposure is real — Anthropic controls the plugin system, the distribution, and the competitive landscape — but the downside scenarios carry embedded upside.

Our depth advantage persists in either scenario. The plugin encodes Amazon's Working Backwards process specifically [11], with reference guides, a peer review framework grounded in Cagan's four risks [4], and a Kahneman-informed decision quality checklist. A general-purpose product-thinking feature is unlikely to replicate that specificity. Users who value the validation of a framework with two decades of track record at Amazon would continue to prefer the specialized tool even if a native alternative exists. The mitigation against platform concentration is explored in the Feature Appendix: alternative CLI-based distribution channels reduce single-platform dependency over time.

Technical

Q14: What are the major technical risks?

For the shipped core (`/prfaq`, `/prfaq:feedback`), technical risks are minimal. The plugin is a set of markdown files plus a LaTeX template and a shell script. No backend, no API, no database. The LaTeX dependency requires users to have a TeX distribution installed, which is a mild friction point; the installer mitigates with platform-specific setup instructions.

The planned `/prfaq:meeting` feature introduces genuine technical risk. Multi-agent orchestration — running distinct persona agents (target customer, principal engineer, unit economics reviewer, UX bar raiser, skeptical executive) with visible debate — depends on Claude Code's ability to manage concurrent agent threads with distinct system prompts. Persona fidelity is the hard problem: each agent must stay in character, disagree substantively (not performatively), and produce debate that is genuinely useful rather than theatrical. If persona quality is not high enough, the feature degrades from "simulated review meeting" to "five versions of the same generic feedback." Mitigation: prototype with two personas first, validate debate quality, then scale to the full panel.

The deeper risk is strategic, not technical. The plugin depends entirely on Claude Code's plugin system for loading, execution, and distribution. Anthropic controls all three. If Anthropic deprecates the plugin API, restricts third-party plugins, or ships a native alternative, the migration cost is real: prompt engineering must be adapted to a new host, distribution must be rebuilt from scratch, and the installed base may not follow. The plugin's architecture — markdown files, a LaTeX template, a shell script — is portable in principle, but retargeting to another CLI-based agent (Codex CLI, Gemini CLI) requires non-trivial adaptation. This platform dependency is why feasibility risk is rated Medium rather than Low.

Q15: What dependencies exist on other teams or systems?

The plugin depends on Anthropic's Claude Code plugin system for loading and execution, and on a local TeX distribution for PDF compilation. Both are external dependencies outside our control. The Claude Code plugin API is stable (v1), and TeX Live is a mature, 30-year-old ecosystem. If Anthropic changes the plugin format, the migration would be straightforward — the plugin is a handful of files with simple structure. If TeX Live is unavailable, the plugin still generates the `.tex` source file; only PDF compilation is affected.

All output is portable. The `.tex` source files and compiled PDFs are standalone artifacts — they do not require the plugin to read, edit, or compile. If the plugin becomes unavailable, users retain their documents and can compile them with any standard TeX distribution. No content is locked inside a proprietary format or dependent on a running service. This portability is a deliberate design choice: the plugin is a workflow tool, not a platform, and the documents it produces belong entirely to the user.

Q16: If this succeeds, what breaks first?

Nothing at the infrastructure level — the plugin is stateless, runs locally, and requires no servers. At scale, the pressures are human, not technical: (1) GitHub issue volume — at 1,000+ users, support requests and feature demands will exceed solo maintainer capacity; mitigation is clear contribution guidelines and aggressive scope discipline (see Won't Do list); (2) prompt maintenance — as Claude models evolve, skill prompts may need tuning to maintain output quality; (3) backwards compatibility — if Anthropic changes the plugin system, the plugin must migrate; the simple file-based architecture makes this low-risk; (4) community expectations — a popular free tool attracts requests for features that contradict the design philosophy (dashboards,

collaboration, web interfaces). The Won't Do list exists to make these decisions in advance, not under pressure.

Q17: What is the estimated development timeline?

The core plugin (`/prfaq` and `/prfaq:feedback`) is built and shipping. Remaining V1 work is `/prfaq:import`, which requires document parsing across multiple input formats (plain text, Markdown, Google Doc paste, raw `.tex`) and gap analysis logic — estimated at one to two development sessions. The V2 feature (`/prfaq:meeting`) is the most technically ambitious: multi-agent orchestration with distinct persona prompts, visible debate threading, and decision synthesis. Estimated at three to five sessions of iterative development and prompt tuning. There is still no infrastructure to build — all complexity is in prompt engineering and skill orchestration.

Business

Q18: What is the revenue model?

There is none. `prfaq` is free and open source under a permissive license. The strategic value is threefold: (1) it establishes punt-labs as the reference implementation for AI-assisted product development tooling — the plugin that defines how structured product thinking works inside an AI coding agent; (2) it builds reputation and visibility in the Claude Code ecosystem; (3) it creates a feedback loop for understanding how builders approach product thinking, which informs other products at punt-labs.

Q19: Is there a commercial model?

Not today, and not without evidence. The plugin is free and will remain free. However, if product-market fit is confirmed through the validation plan (see FAQ 12), there is a natural premium tier: subscription access to professional research sources. The free plugin uses web search to find evidence for claims in the PR/FAQ. A premium research tier could provide access to professional databases — Pitchbook for market sizing, Statista for industry statistics, proprietary industry reports — as well as AI-powered survey agents that conduct lightweight customer discovery interviews on the user's behalf. These capabilities address the weakest link in most PR/FAQs: the evidence base. A builder who can generate a PR/FAQ with real market data and structured customer signals has a meaningfully better decision document than one relying on free web search alone.

This is a path, not a plan. The decision to build a commercial tier is gated on three conditions: (1) the free plugin achieves PMF — repeat usage, shareability, and community traction as defined in FAQ 12; (2) user feedback specifically identifies research quality as a bottleneck; (3) the unit economics of licensing professional data sources close at a price point the target audience will pay. Until all three conditions are met, the commercial model remains a documented option, not a roadmap item.

Q20: What does the cost structure look like at steady state?

`prfaq` has zero infrastructure cost — no servers, no databases, no APIs. The real cost is maintainer time. At current scale (pre-launch): approximately 2 hours per week on development and prompt refinement. At the 500-star target: an estimated 4–8 hours per week covering issue triage, community engagement, prompt maintenance as Claude models evolve, and occasional

LaTeX template updates. At 1,000+ users: maintainer capacity becomes the constraint. The plugin is designed to be self-sustaining: the entire codebase is 9 files with no infrastructure dependencies, making it trivially forkable. If maintainer capacity is exceeded, the community can maintain it independently. At that scale, the project either recruits community co-maintainers or narrows support scope to bug fixes only. The opportunity cost is hours not spent on other punt-labs projects. This is acceptable while the plugin establishes punt-labs as the reference implementation for AI-assisted product development tooling; it should be re-evaluated if other punt-labs priorities require the same hours.

Q21: What are the key metrics for success?

Three metrics, measured via GitHub analytics and community signals:

1. **GitHub stars and forks** — target: 500 stars within 6 months. This measures discoverability and perceived value.
2. **Issues and pull requests from external contributors** — target: 10 external PRs within 6 months. This measures whether the plugin is useful enough that people invest time improving it.
3. **Mentions in builder and developer communities** (Hacker News, Twitter/X, Reddit, Discord servers) — target: 3 organic mentions within 3 months. This measures whether the plugin solves a real problem that people tell others about.

If none of these targets are met after 6 months, the plugin is not reaching its audience and the distribution strategy should be reconsidered.

Q22: Why now? What has changed?

Three shifts converged. First, AI coding tools reached mainstream adoption in 2025–2026, creating a large population of engineers and technical founders who can build products independently but lack product discipline. Second, Claude Code's plugin system matured, making it possible to deliver structured workflows inside the developer's existing tool. Third, the consequences of "vibe coding without product thinking" are becoming visible — failed launches, rebuilds, wasted months — creating demand for lightweight product frameworks that do not require hiring a PM or leaving the terminal. A fourth tailwind is emerging: Claude Code is crossing over to non-technical users [5], which expands the potential audience beyond the initial technical core.

Q23: What is the customer acquisition strategy?

Distribution is the product strategy for a free plugin. Primary channels: (1) GitHub discoverability — README, topics, and stars drive organic search traffic; (2) community seeding — posts on Hacker News, Reddit r/ClaudeAI, and Twitter/X targeting the "vibe coding" and AI-builder audience; (3) content marketing — the plugin's own PR/FAQ serves as a case study and demo artifact; (4) word-of-mouth — builders who find the plugin useful share the PDF output, which includes a link back to the repository. There is no paid acquisition.

The secondary audience — non-technical Claude Code users [5] — is not actively targeted in V1. They discover the plugin organically through Claude Code's plugin ecosystem and through technical users who share the PDF output. This is a spillover, not an acquisition strategy.

Estimated time investment: 2–4 hours per week on community engagement for the first 3 months. Conversion assumption: a front-page Hacker News post reaches approximately 20–30K views and converts at roughly 0.1–0.2%, yielding around 20–60 installs per post. These numbers

are testable hypotheses, not forecasts. If organic channels do not work within 6 months, the distribution hypothesis is wrong and should be revisited.

Q24: What are we not building?

We are not building a SaaS product management platform. We are not building a competitor to Jira, Linear, Productboard, or Notion. We are not adding collaboration features, dashboards, or analytics. We are not building a web interface. The plugin is a single-player tool that runs locally, produces a document, and gets out of the way. Scope discipline is the feature. See the Feature Appendix for the full Must Do / Should Do / Won't Do breakdown.

Q25: It is one year from now and prfaq failed. What went wrong?

Most likely failure scenario: builders do not want product discipline, even when it is free and frictionless. The vibe coding audience builds for the joy of building, and a tool that asks "what evidence do you have that customers want this?" feels like a buzzkill, not a feature. Second scenario: the plugin does not surface in a crowded marketplace — the Claude Code plugin ecosystem is growing rapidly, and discoverability is poor. If users cannot find `prfaq` among hundreds of alternatives, organic adoption stalls regardless of quality. Third scenario: the output quality is not high enough to share with stakeholders — if the generated PR/FAQ reads like AI slop rather than a credible decision document, users will not trust it for real decisions. Fourth scenario: the PR/FAQ process is too heavyweight for the target audience — builders want a quick sanity check, not a formal document, and a lighter-weight alternative captures the market. Fifth scenario: scope overreach — the planned meeting simulation (`/prfaq:meeting`) requires multi-agent orchestration and persona fidelity that proves harder than expected, consuming maintainer time that should go toward polishing the core generation and iteration workflow. Sixth scenario: we misidentified the customer or their motivation — the tool solves the wrong person's problem, or targets the right person for the wrong reason. Builders may want stakeholder theater (a polished-looking document that signals rigor) rather than genuine product discipline, and when `prfaq` forces real answers to hard questions, they abandon it for something that produces impressive artifacts without the uncomfortable introspection.

Risk Assessment

Risk	Rating	Assessment
Value	Medium	Market-level evidence strong (proven framework, large technical audience). Customer-level evidence absent — no interviews confirming demand for this delivery mechanism from engineers and technical founders. Plugin is free, so adoption barrier is low. Mitigation: 10–20 person validation trial with opt-in metrics and structured feedback (see FAQ 12).
Usability	Low	One-line install, one slash command, Claude guides interactively. TeX dependency is the main friction; installer mitigates. Time to value under one hour.
Feasibility	Medium	Core plugin built and functional. No backend, no infrastructure. V1 remaining work (`/prfaq:import`) is straightforward parsing. V2 meeting simulation carries localized execution risk in multi-agent persona fidelity. The strategic risk is platform dependency: the plugin runs exclusively on Claude Code, and Anthropic controls the plugin system, distribution, and competitive response. If Anthropic deprecates the plugin API or ships a native alternative, migration cost is non-trivial. Mitigation: portable LaTeX output (users retain documents regardless of plugin availability), exploration of alternative CLI-based distribution channels, and architecture that minimizes platform-specific coupling.
Viability	Medium	Zero infrastructure cost but maintainer time is the binding constraint. At 500+ stars, estimated 4–8 hrs/week on issues, community, and prompt maintenance. The plugin is designed to survive maintainer abandonment: 9 files, no backend, standalone LaTeX output. An engineer-heavy user base naturally tracks potential co-maintainers. Mitigation: contribution guidelines, scope discipline, forkable architecture.

Feature Appendix

Must Do (V1)

- **F1.** **Interactive discovery workflow** — structured questions that guide the user from idea to document
- **F2.** **Complete PR/FAQ document generation** — press release, external FAQs, internal FAQs, four risks assessment
- **F3.** **LaTeX compilation to professional PDF** — shareable artifact, not a disposable brainstorm
- **F4.** **Reference guides** — encoded best practices for PR structure, FAQ structure, four risks, common mistakes, unit economics, feasibility, and usability
- **F5.** **Peer review agent** — automated critique against Working Backwards principles and cognitive bias checklist
- **F6.** **Research sub-agent** — scans local `./research/` directory, indexed documents, and the web for evidence
- **F7.** **Biblatex citation system** — academic-style citations for claims, linking evidence to assertions
- **F8.** **`/prfaq:feedback`** — directed iteration from pointed user feedback; traces cascading effects via section dependency graph and surgically redrafts affected sections
- **F9.** **`/prfaq:import`** — ingest an existing PR/FAQ draft (plain text, Markdown, or raw `.tex`), parse and structure it against the framework, research evidence for unsupported claims, format into the LaTeX template with proper environments and citations, compile to PDF, and present a gap analysis

Should Do (V2)

- **F10.** **`/prfaq:meeting`** — simulate an Amazon-style PR/FAQ review meeting. The document is reviewed page by page, FAQ by FAQ, with agentic personas (target customer, principal engineer, unit economics reviewer, UX bar raiser, skeptical executive). Agent debate is visible to the user. Main Claude facilitates and synthesizes debate into pointed decision questions. The user is the PM and final decision-maker. Decisions feed into `/prfaq:feedback` for automated iteration. This is the most ambitious planned feature and carries meaningful execution risk in multi-agent orchestration and persona fidelity.
- **F11.** **Markdown output mode** — alternative to LaTeX for users without a TeX distribution
- **F12.** **Comparative analysis mode** — evaluate two competing product ideas side-by-side
- **F13.** **Template customization** — allow users to override the LaTeX template for branding
- **F14.** **`/prfaq:feedback-to-us`** — opt-in command that lets users send structured feedback (what worked, what did not, whether the output was shared) via a single API call. Reduces friction of collecting qualitative data from terminal-native users. Built only if Phase 1 validation trials (see FAQ 12) confirm that users want to give feedback but existing channels are too cumbersome.
- **F15.** **Alternative CLI-based distribution channels** — explore Codex CLI, Gemini CLI, and other terminal-based AI coding tools as additional distribution surfaces. The plugin's architecture — markdown skill files, a LaTeX template, and a shell script — is not deeply coupled to Claude Code's plugin system. Porting to other CLI-based agents would reduce single-platform concentration risk and expand reach to builders who use different tools. This is not about supporting multiple LLM backends simultaneously; it is about ensuring the product-thinking workflow is available wherever builders work in terminals. Priority increases

if Claude Code's plugin ecosystem becomes restrictive or if a competing CLI tool gains significant share among the target audience.

Won't Do

- **F16.** **Web dashboard or SaaS platform** — the plugin runs locally inside Claude Code. Adding a web layer would introduce infrastructure, authentication, and hosting costs that contradict the zero-ops design. If users want to share documents, they share the PDF.
- **F17.** **Collaboration or multi-user editing** — PR/FAQ is a single-author process by design. The document is written by one person, then reviewed by others. Real-time collaboration would encourage design-by-committee, which the Working Backwards process explicitly avoids.
- **F18.** **Project management features** — no task tracking, no roadmap views, no sprint planning. The plugin produces a decision document, not a project plan. Users who need project management have Linear, Jira, or beads.

References

[1] Stack Overflow. *2025 Developer Survey*. Annual survey of 65,000+ developers on tools, practices, and AI adoption. 2025. URL: https://survey.stackoverflow.co/2025.

[2] Andrej Karpathy. *Vibe Coding*. Original post coining the term "vibe coding" — viewed over 4.5 million times. Feb. 2025. URL: https://x.com/karpathy/status/1886192184808149383.

[3] HFS Research and Publicis Sapient. *Smash Through Tech Debt: Why AI Is the Jackhammer*. Survey of 608 IT and business leaders estimating Global 2000 accumulated technical debt at $1.5–2 trillion. 2025. URL: https://www.hfsresearch.com/research/publicis-sapient-jackhammer/.

[4] Marty Cagan. *Inspired: How to Create Tech Products Customers Love*. 2nd ed. Wiley, 2018. ISBN: 978-1119387503.

[5] Natallie Rocha. "This A.I. Tool Is Going Viral. Five Ways People Are Using It." In: *The New York Times* (Jan. 2026). Profiles five non-programmers using Claude Code: school administrator, photographer, prosecutor, finance professor, welding business owner. URL: https://www.nytimes.com/2026/01/23/technology/claude-code.html.

[6] TechCrunch. *Lovable Says It's Nearing 8 Million Users*. Lovable, an AI coding startup founded in 2024, nearing 8 million users within one year. Nov. 2025. URL: https://techcrunch.com/2025/11/10/lovable-says-its-nearing-8-million-users-as-the-year-old-ai-coding-startup-eyes-more-corporate-employees/.

[7] Amjad Masad. *Replit CEO: We Don't Care About Professional Coders Anymore*. Replit's 35M+ users are 75% non-coders; CEO explicitly pivoting away from professional developers. Jan. 2025. URL: https://www.analyticsvidhya.com/blog/2025/01/replit-ceo-we-dont-care-about-professional-coders-anymore/.

[8] Sacra. *Bolt.new Revenue, Funding & News*. Bolt.new hit $40M ARR in March 2025, progressing from $4M ARR within 4 weeks of launch. 2025. URL: https://sacra.com/c/bolt-new/.

[9] Sacra. *Cursor Revenue, Valuation & Funding*. Cursor reached $1B ARR and 1M daily active users. 2025. URL: https://sacra.com/c/cursor/.

[10] TechCrunch. *A Quarter of Startups in YC's Current Cohort Have Codebases Almost Entirely AI-Generated*. 25% of Y Combinator W25 cohort startups have codebases that are almost entirely AI-generated. Mar. 2025. URL: https://techcrunch.com/2025/03/06/a-quarter-of-startups-in-ycs-current-cohort-have-codebases-that-are-almost-entirely-ai-generated/.

[11] Colin Bryar and Bill Carr. *Working Backwards: Insights, Stories, and Secrets from Inside Amazon*. St. Martin's Press, 2021. ISBN: 978-1250267597.

[12] Constellation Research. *Anthropic's Claude Code Revenue Doubled Since Jan. 1*. Claude Code annualized run-rate revenue reached approximately $2.5B as of February 2026, more than doubling since January 1. Feb. 2026. URL: https://www.constellationr.com/insights/news/anthropics-claude-code-revenue-doubled-jan-1.